Introduction to Machine Learning

Decision Trees

Giacomo Fiumara

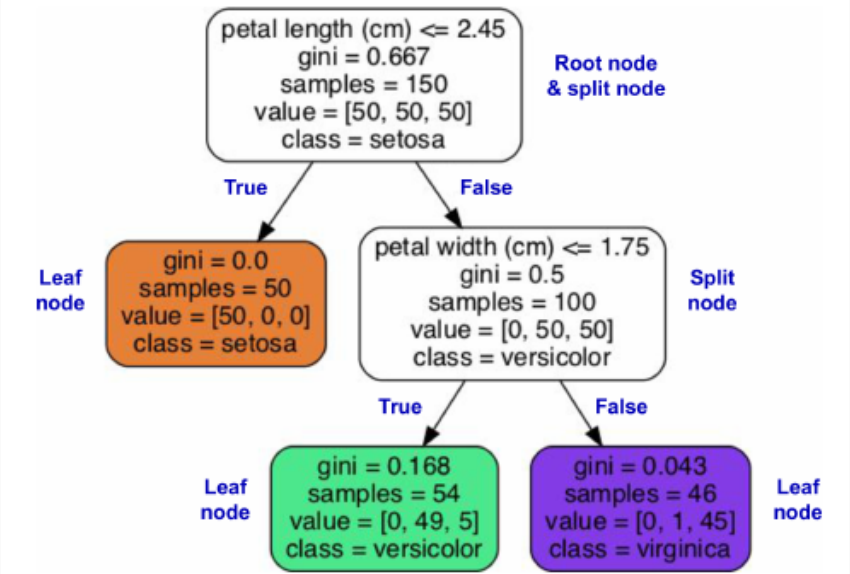
Decision Trees: Overview

- Decision trees are supervised learning models usable for classification, regression, and multioutput predictions.
- They can fit very complex datasets and are the basic building block of random forests and other tree ensembles.
- The chapter covers training and visualization, the CART algorithm, regularization, regression trees, and limitations of decision trees.
- Compared to many models, decision trees require very little feature preprocessing; no scaling or centering is needed.

Training a Decision Tree on Iris

- Example dataset: Iris, using only two features for visualization:
- Petal length (cm) and petal width (cm) are used as input features, and the Iris species is the target class.
- A `DecisionTreeClassifier` is trained with a small maximum depth, for example `max_depth = 2`.
- The trained tree is exported using `export_graphviz` and visualized with Graphviz (e.g., as PDF or PNG).

Iris Decision Tree Structure



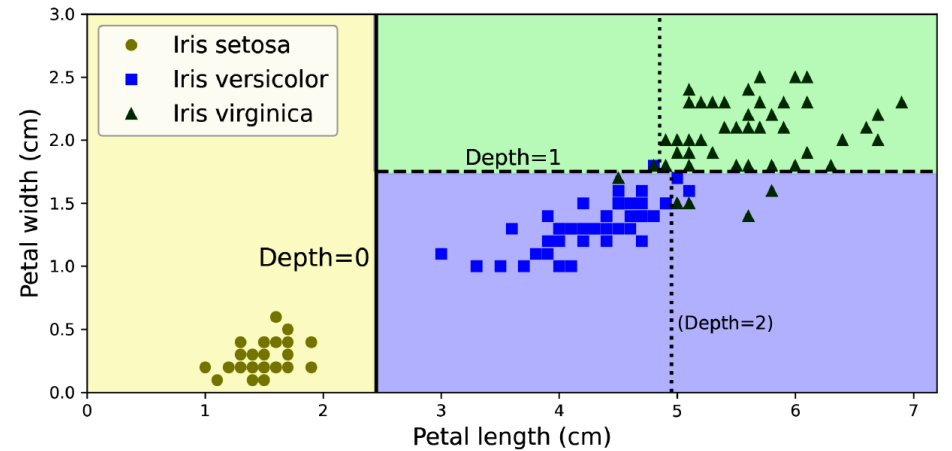
Iris Decision Tree: Interpretation

- Each internal node represents a test on a feature, with a numeric threshold (e.g., petal length smaller than a value).
- Each branch corresponds to the outcome of the test, and each leaf node stores a predicted class for the instances that reach it.
- Node annotations typically include the impurity measure, the number of samples, and the class distribution.
- This tree for Iris is very shallow, which makes its decision process easy to explain and to visualize.

Making Predictions with a Decision Tree

- To classify a new instance, start at the root node and follow the path determined by the feature tests.
- At each internal node, compare the instance's feature value with the node's threshold and go left or right accordingly.
- Once a leaf node is reached, the class associated with that leaf is the predicted class for the instance.
- In the Iris example, short petals lead directly to a leaf classifying flowers as *Iris setosa*.

Decision Boundaries in Feature Space



Decision Boundaries: Interpretation

- The tree partitions the feature space into rectangular regions separated by axis-aligned decision boundaries.
- Each split creates a boundary such as "petal length $\leq$ threshold" or "petal width $\leq$ threshold".
- Regions containing only one class are pure and are not split further when depth or impurity constraints are met.
- Increasing the maximum depth adds more boundaries, allowing a finer partition of the feature space.

Node Attributes: Samples, Class Counts, Impurity

- Each node stores the number of training instances that reach it; this count is often labeled as `samples`.
- The vector of class counts for that node is sometimes labeled `value`, one entry per class.
- An impurity measure (often Gini impurity) quantifies how mixed the node is with respect to the target classes.
- A node is pure when all training instances at that node belong to the same class, which corresponds to zero impurity.

Gini Impurity: Definition

- The Gini impurity of node i is defined in terms of the class proportions at that node.
- Let $p_{i,k}$ be the proportion of class k instances among the training samples that reach node i.
- The Gini impurity G_i is:
- $$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$
- A pure node has $G_i = 0$, while mixed nodes have higher values of Gini impurity.

Gini Impurity: Example Computation

- Consider a node that receives instances from three classes with counts c_1, c_2, c_3.
- The total number of instances at the node is:

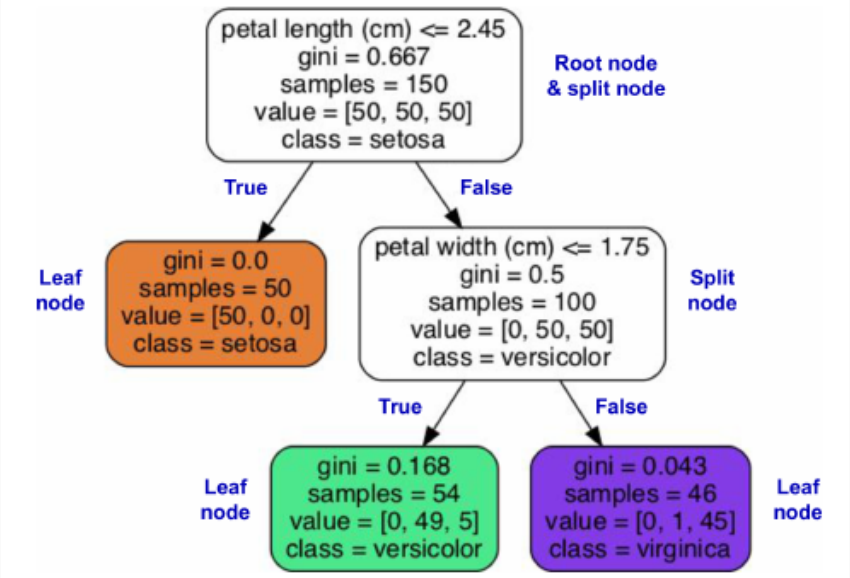
$$m = c_1 + c_2 + c_3$$

- The class proportions are:

$$p_{i,1} = \frac{c_1}{m}, \quad p_{i,2} = \frac{c_2}{m}, \quad p_{i,3} = \frac{c_3}{m}$$

- Substituting these proportions into the formula yields the Gini impurity for that node.

Iris Decision Tree Structure



Making Predictions

- (Root node) `[50, 50, 50]`

$$G = 1 - 3\left(\frac{50}{150}\right)^2 = 0.66667$$

- (Level 1 - Left node) `[50, 0, 0]`

$$G = 1 - \left(\left(\frac{50}{50}\right)^2 + \left(\frac{0}{50}\right)^2 + \left(\frac{0}{50}\right)^2\right) = 0$$

- (Level 1 - Right node) `[0, 50, 50]`

$$G = 1 - \left(\left(\frac{0}{100}\right)^2 + \left(\frac{50}{100}\right)^2 + \left(\frac{50}{100}\right)^2\right) = 0.5$$

- (Level 2 - Left node) `[0, 49, 5]`

$$G = 1 - \left(\left(\frac{0}{54}\right)^2 + \left(\frac{49}{54}\right)^2 + \left(\frac{5}{54}\right)^2\right) = 0.168$$

- (Level 2 - Right node) `[0, 1, 45]`

$$G = 1 - \left(\left(\frac{0}{46}\right)^2 + \left(\frac{1}{46}\right)^2 + \left(\frac{45}{46}\right)^2\right) = 0.042$$

White-Box Models and Interpretability

- Decision trees are often called white-box models because their structure is directly interpretable by humans.
- The sequence of tests along a path from root to leaf is a transparent decision rule that can be read and applied manually.
- In contrast, many powerful models such as random forests and neural networks are considered black-box models.
- Interpretable models are especially important in applications where explanations and fairness are critical.

Estimating Class Probabilities

- A decision tree can estimate, for each leaf, the probability that an instance belongs to each class.
- To predict probabilities, the tree first routes the instance to a leaf node using the usual splitting tests.
- At the leaf, the class probabilities are obtained by normalizing the class counts for the training instances in that leaf.
- The predicted class for the instance is then the class with the highest estimated probability at the leaf.

The CART Training Algorithm

- Scikit-Learn trains decision trees using the CART algorithm (Classification and Regression Trees).
- At each step, CART splits the current set into two subsets using a feature index k and a threshold t_k.
- The split is chosen to produce child nodes that are as pure as possible, according to an impurity or error measure.
- This process is applied recursively: each subset is split again until stopping conditions are met.

CART Cost Function for Classification

- For classification, CART chooses a split (k, t_k) by minimizing a weighted impurity of the two child nodes.
- Let m be the number of instances at the current node, and m_{left}, m_{right} the sizes of the child subsets.
- The cost function that CART minimizes is:
- $$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$
- Here G_{left} and G_{right} are the impurity measures (e.g., Gini) of the left and right child nodes.

Greedy Nature and Optimality

- CART is a greedy algorithm: it optimizes the split locally at each node without looking ahead several levels.
- It does not search for a globally optimal tree: it does not backtrack once a split is chosen.
- Finding the globally optimal decision tree is an NP-complete problem and is intractable for realistic dataset sizes.
- In practice, the greedy procedure usually finds a tree that performs well, even if it is not optimal.

Prediction Complexity

- Making a prediction with a decision tree requires following a path from the root to a leaf.
- For a reasonably balanced tree, the depth is on the order of $\log_2(m)$, where m is the number of training instances.
- Each node test involves checking a single feature value against a threshold.
- Overall, prediction time complexity is approximately $O(\log_2(m))$, independent of the number of features.

Training Complexity

- At each node, the algorithm considers splits on all features (or on a subset if `max_features` is set).
- For each feature, instances are evaluated to find the best threshold that minimizes the chosen cost function.
- For a tree grown on m instances and n features, the training complexity is roughly:

$$O(n \times m \log_2(m))$$

- This makes decision tree training scalable to fairly large datasets, especially when features are not too numerous.

Gini Impurity Versus Entropy

- For classification, Scikit-Learn supports two main impurity measures: Gini impurity and entropy.
- Gini impurity is the default in `DecisionTreeClassifier`; entropy can be selected by setting `criterion="entropy"`.
- Both measures quantify how mixed the classes are at a node: lower values indicate purer nodes.
- In many practical problems, using Gini or entropy leads to very similar trees and performance.

Entropy: Definition

- Entropy comes from information theory and measures the average information content of a random variable.
- For node i, with class proportions $p_{i,k}$, the entropy H_i is defined as:

$$H_i = -\sum_{k=1}^{n} p_{i,k} \log_2 p_{i,k}$$

- Entropy is zero when all instances at the node belong to a single class and larger when the distribution is more mixed.

Practical Choice: Gini or Entropy?

- Gini impurity is slightly faster to compute than entropy, so it is often chosen as the default criterion.
- When the two criteria differ, Gini tends to isolate the most frequent class in a branch of the tree.
- Entropy sometimes produces slightly more balanced trees, but the performance difference is usually small.
- In practice, either measure is acceptable; Gini impurity is a good default starting point.

Overfitting and Nonparametric Nature

- Unconstrained decision trees can grow very deep and complex, fitting noise and idiosyncrasies of the training data.
- Such trees are powerful but prone to severe overfitting, especially when data is noisy or limited.
- Decision trees are called nonparametric models because their effective number of parameters grows with the data.
- In contrast, parametric models (such as linear models) have a fixed number of parameters chosen before training.

Regularization via Tree Shape Constraints

- To control overfitting, decision tree training must be regularized using constraints on the tree structure.
- Limiting the maximum depth of the tree is one of the most common regularization strategies.
- In Scikit-Learn, the `max_depth` hyperparameter sets the maximum allowed depth of the tree.
- Smaller `max_depth` values generally reduce variance and overfitting but may increase bias and underfitting.

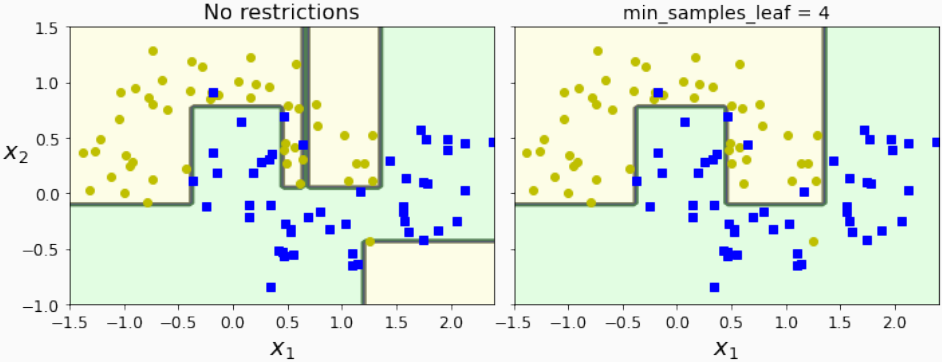
Additional Regularization Hyperparameters

- Other hyperparameters control when and how nodes are split or leaves are created.
- `max_features`: maximum number of input features evaluated for each split.
- `max_leaf_nodes`: maximum number of leaf nodes in the tree.
- `min_samples_split`: minimum number of instances a node must have before it is allowed to split.
- `min_samples_leaf`: minimum number of instances required to be at a leaf node.
- `min_weight_fraction_leaf`: similar to `min_samples_leaf`, expressed as a fraction of the total sample weight.

Pruning and Statistical Tests (Conceptual)

- Some decision tree algorithms first grow a large tree, then prune it by removing nodes that do not generalize well.
- A split can be considered unnecessary if the purity gain it provides is not statistically significant.
- Statistical tests, such as chi-squared tests, can be used to decide whether a node should be pruned.
- Pruning continues until all remaining splits are supported by significant impurity reductions.

Effect of Regularization: Example



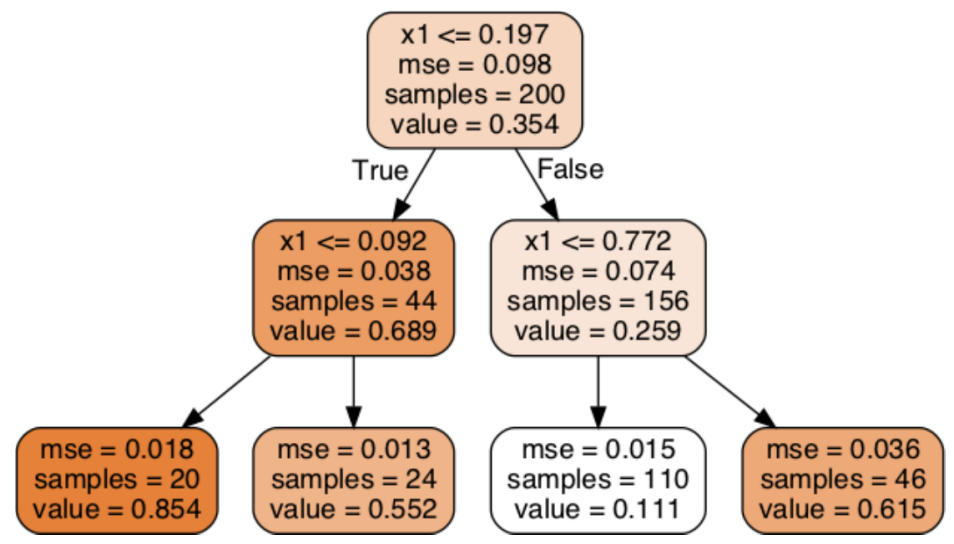
Effect of Regularization: Interpretation

- On complex datasets, an unregularized tree can produce extremely irregular decision boundaries and overfit the training set.
- Applying regularization (for example via `min_samples_leaf`) smooths the boundaries and improves generalization.
- The regularized tree may achieve higher accuracy on a test set, even if training accuracy is slightly lower.
- Choosing regularization hyperparameters is typically done using validation sets or cross-validation.

Decision Trees for Regression

- Decision trees can also model continuous target values using `DecisionTreeRegressor`.
- The tree structure is similar: internal nodes perform threshold tests on input features, and leaves store predicted values.
- At each leaf, the predicted value is the average of the target values of the training instances that reach that leaf.
- The training algorithm chooses splits that minimize a regression error measure instead of a classification impurity.

Example Regression Tree



Regression Tree Predictions

- For a regression tree, the feature space is partitioned into regions, and each region is assigned a constant prediction.
- The constant value in each region is the mean of the target values of the training samples that fall in that region.
- The training algorithm chooses splits that make the samples in each region close to this mean, according to a loss measure.
- Deeper regression trees can represent more complex functions but risk overfitting, just as in classification.

CART Cost Function for Regression

- For regression, CART minimizes a weighted sum of mean squared errors in the left and right child nodes.
- For a candidate split (k, t_k), the cost function is:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

- Here MSE_{left} and $\text{MSE}_{\text{right}}$ are the mean squared errors of the target values in the left and right subsets.

Mean Squared Error at a Node

- For a node, with target values y_i for the instances that reach it, the mean target value is:

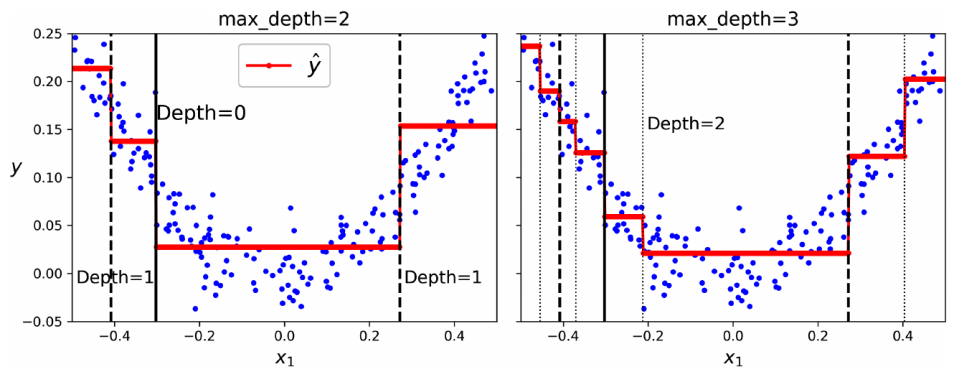
$$\bar{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y_i$$

- The mean squared error at that node is:

$$\text{MSE}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} (y_i - \bar{y}_{\text{node}})^2$$

- Splits are chosen to reduce the weighted sum of these node MSE values as much as possible.

Overfitting in Regression Trees



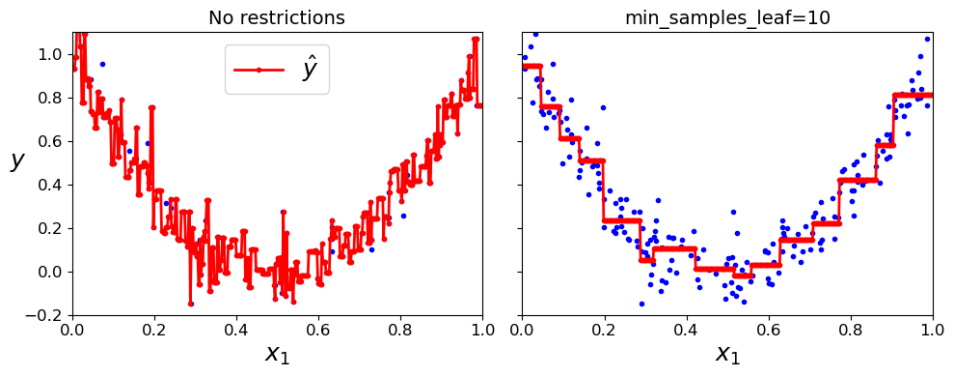
Regularizing Regression Trees

- Without constraints, regression trees produce very jagged prediction curves, closely matching the training data.
- This leads to high variance and poor generalization to new instances.
- Regularization techniques such as increasing `min_samples_leaf` or limiting `max_depth` smooth the predictions.
- A properly regularized regression tree trades some training accuracy for better test performance.

Sensitivity to Axis Orientation

- Decision trees split using axis-aligned thresholds, so decision boundaries are always perpendicular to feature axes.
- This makes them sensitive to the orientation of the data in feature space.
- A simple rotation of the dataset can turn an easy classification problem into one that requires many more splits.
- As a result, some decision trees may become unnecessarily complex after such transformations.

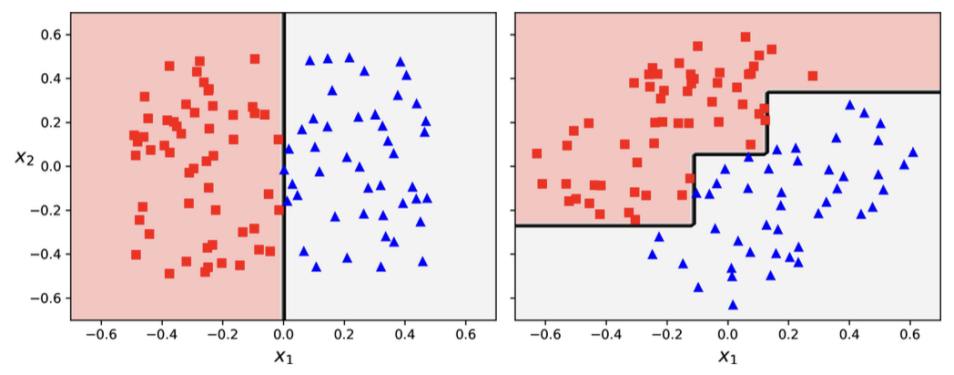
Effect of Rotation on Decision Boundaries



Mitigating Orientation Sensitivity

- One way to mitigate sensitivity is to preprocess the data using techniques that decorrelate or rotate the features.
- Standardization followed by principal component analysis (PCA) can often reduce redundant directions.
- After such transformations, a decision tree may approximate the decision boundary with fewer, simpler splits.
- Nevertheless, tree splits remain axis-aligned in the transformed feature space.

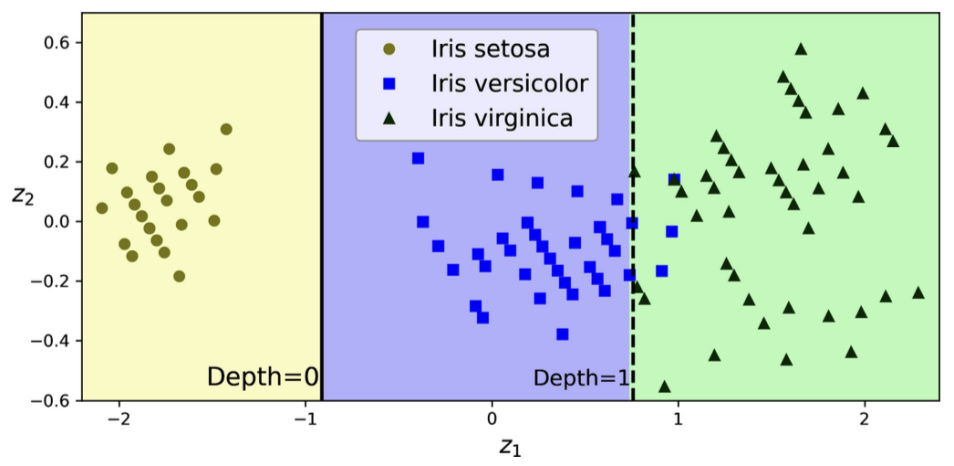
Decision Boundaries After PCA



High Variance of Decision Trees

- Decision trees are high-variance models: small changes in the training set can produce very different trees.
- The training algorithm is often stochastic, for example by selecting random subsets of features at each split.
- Even when trained on the same dataset, retraining with a different random seed can result in different tree structures.
- This variability is a weakness of single trees, but it becomes a strength in ensemble methods.

Example of Variability in Trees



From Single Trees to Random Forests

- A random forest is an ensemble of decision trees trained on different subsets of the data and features.
- Each tree makes its own prediction, and the ensemble combines them, for example by majority vote for classification.
- Averaging predictions reduces the variance of the model and typically improves generalization performance.
- Random forests are among the most powerful and widely used tree-based models in practice.

Conclusions

- Decision trees support both classification and regression, are easy to interpret, and require little preprocessing.
- The CART algorithm greedily grows binary trees, using impurity or error measures and regularization hyperparameters.
- Trees can overfit, are sensitive to axis orientation, and exhibit high variance; ensembles such as random forests address these issues.